

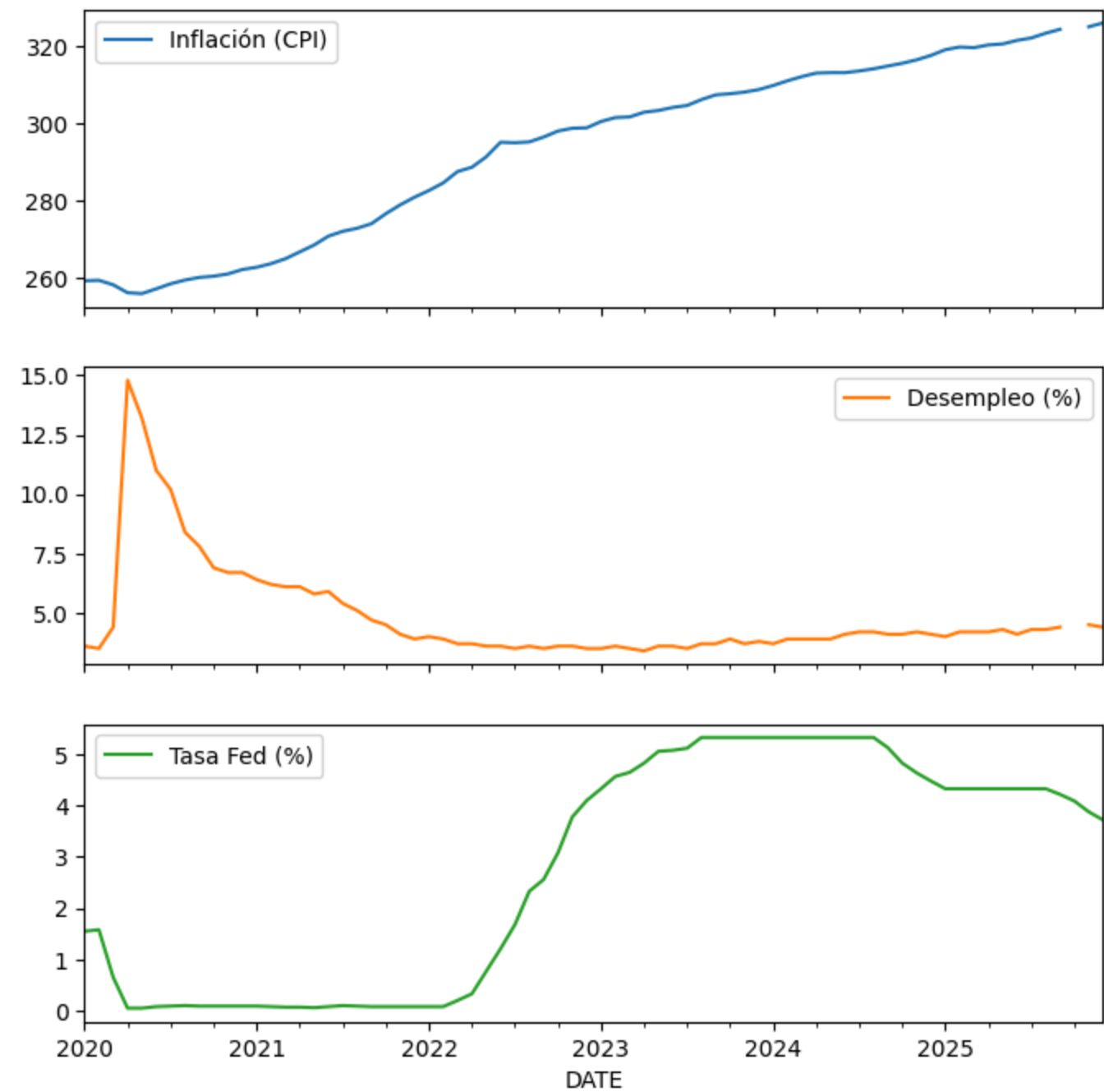
```
1 import pandas_datareader.data as web
2 import datetime
3 import pandas as pd
4
5 # Definir fechas
6 start = datetime.datetime(2020, 1, 1)
7 end = datetime.datetime.today()
8 print("Descargando datos macroeconómicos de FRED")
9
10 # Códigos de Series en FRED (Son universales):
11 # 'CPIAUCSL': Consumer Price Index (Inflación EE.UU.)
12 # 'UNRATE': Unemployment Rate (Tasa de Desempleo)
13 # 'FEDFUNDS': Effective Federal Funds Rate (Tasa de interés Fed)
14 # 'GDP': Gross Domestic Product
15
16 series_codes = ['CPIAUCSL', 'UNRATE', 'FEDFUNDS']
17
18 try:
19     macro_data = web.DataReader(series_codes, 'fred', start, end)
20     macro_data.columns = ['Inflación (CPI)', 'Desempleo (%)', 'Tasa Fed (%)']
21     print("✅ Datos descargados correctamente:")
22     print(macro_data.tail())
23
24     import matplotlib.pyplot as plt
25     macro_data.plot(subplots=True, figsize=(8, 8), title="Datos Macro de FRED")
26     plt.show()
27
28 except Exception as e:
29     print(f"Error al conectar con FRED: {e}")
```

Descargando datos macroeconómicos de FRED

✅ Datos descargados correctamente:

	Inflación (CPI)	Desempleo (%)	Tasa Fed (%)
DATE			
2025-08-01	323.364	4.3	4.33
2025-09-01	324.368	4.4	4.22
2025-10-01	NaN	NaN	4.09
2025-11-01	325.031	4.5	3.88
2025-12-01	326.030	4.4	3.72

Datos Macro de FRED



```
1 import yfinance as yf
2 tickers = ['SPY', 'SCCO', 'GLD']
3 market_data = yf.download(tickers, start='2020-01-01', end=None)['Close']
```

```
4 print(market_data.tail())
```

```
/tmp/ipython-input-2940052452.py:3: FutureWarning: YF.download() has changed argument auto_adjust default to True
  market_data = yf.download(tickers, start='2020-01-01', end=None)['Close']
[*****100%*****] 3 of 3 completed
Ticker      GLD      SCCO
Date
2026-01-23  458.000000  184.300003  689.229980
2026-01-26  464.700012  188.410004  692.729980
2026-01-27  476.100006  194.839996  695.489990
2026-01-28  494.559998  198.179993  695.419983
2026-01-29  495.899994  207.929993  694.039978
```

```
1 monthly_returns = market_data.resample('M').last().pct_change() * 100
2 print(monthly_returns.tail())
```

```

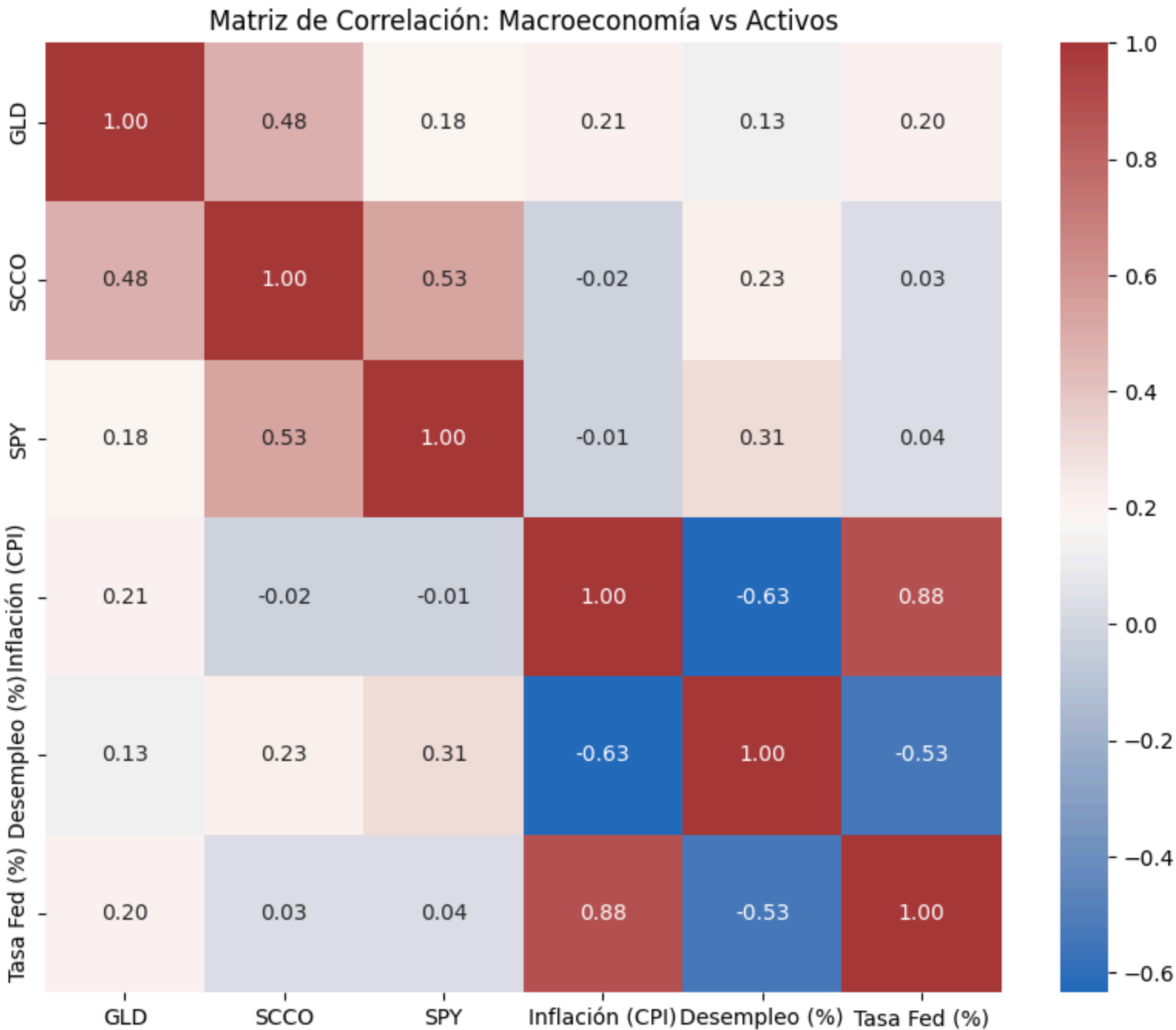
Ticker      GLD      SCCO      SPY
Date
2025-09-30  11.758416  26.298267  3.562037
2025-10-31   3.558667  14.370463  2.383746
2025-11-30   5.367818  -1.479930  0.194993
2025-12-31   2.173351   6.447546  0.079750
2026-01-31  25.129317  44.929247  1.777334
/tmp/ipython-input-3618650109.py:1: FutureWarning: 'M' is deprecated and will be removed in a future version
  monthly_returns = market_data.resample('M').last().pct_change() * 100
```

```
1 # Limpieza rápida
2 macro_data.index = macro_data.index + pd.offsets.MonthEnd(0)
3 monthly_returns.index = monthly_returns.index + pd.offsets.MonthEnd(0)
4
5 # Unir las tablas (concat)
6 master_df = pd.concat([monthly_returns, macro_data], axis=1).dropna()
7
8 print("TABLA MAESTRA (Macro + Mercado):")
9 print(master_df.tail())
```

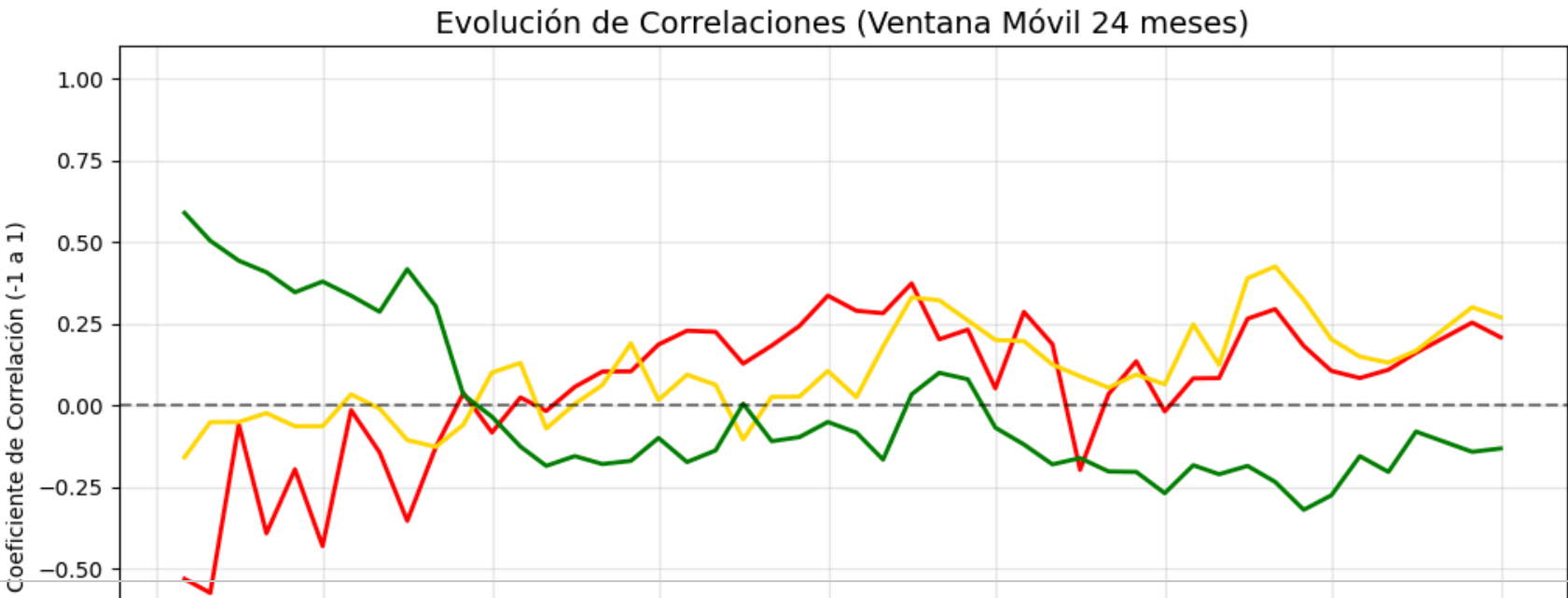
```
TABLA MAESTRA (Macro + Mercado):
      GLD      SCCO      SPY  Inflación (CPI)  Desempleo (%) \
2025-07-31 -0.613455 -6.928930  2.303151      322.132        4.3
2025-08-31  4.987462  3.921756  2.051959      323.364        4.3
2025-09-30  11.758416  26.298267  3.562037      324.368        4.4
2025-11-30  5.367818 -1.479930  0.194993      325.031        4.5
2025-12-31  2.173351  6.447546  0.079750      326.030        4.4

      Tasa Fed (%)
2025-07-31      4.33
2025-08-31      4.33
2025-09-30      4.22
2025-11-30      3.88
2025-12-31      3.72
```

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 # Crear mapa de calor
5 plt.figure(figsize=(10, 8))
6 sns.heatmap(master_df.corr(), annot=True, cmap='vlag', fmt=".2f")
7 plt.title("Matriz de Correlación: Macroeconomía vs Activos")
8 plt.show()
```

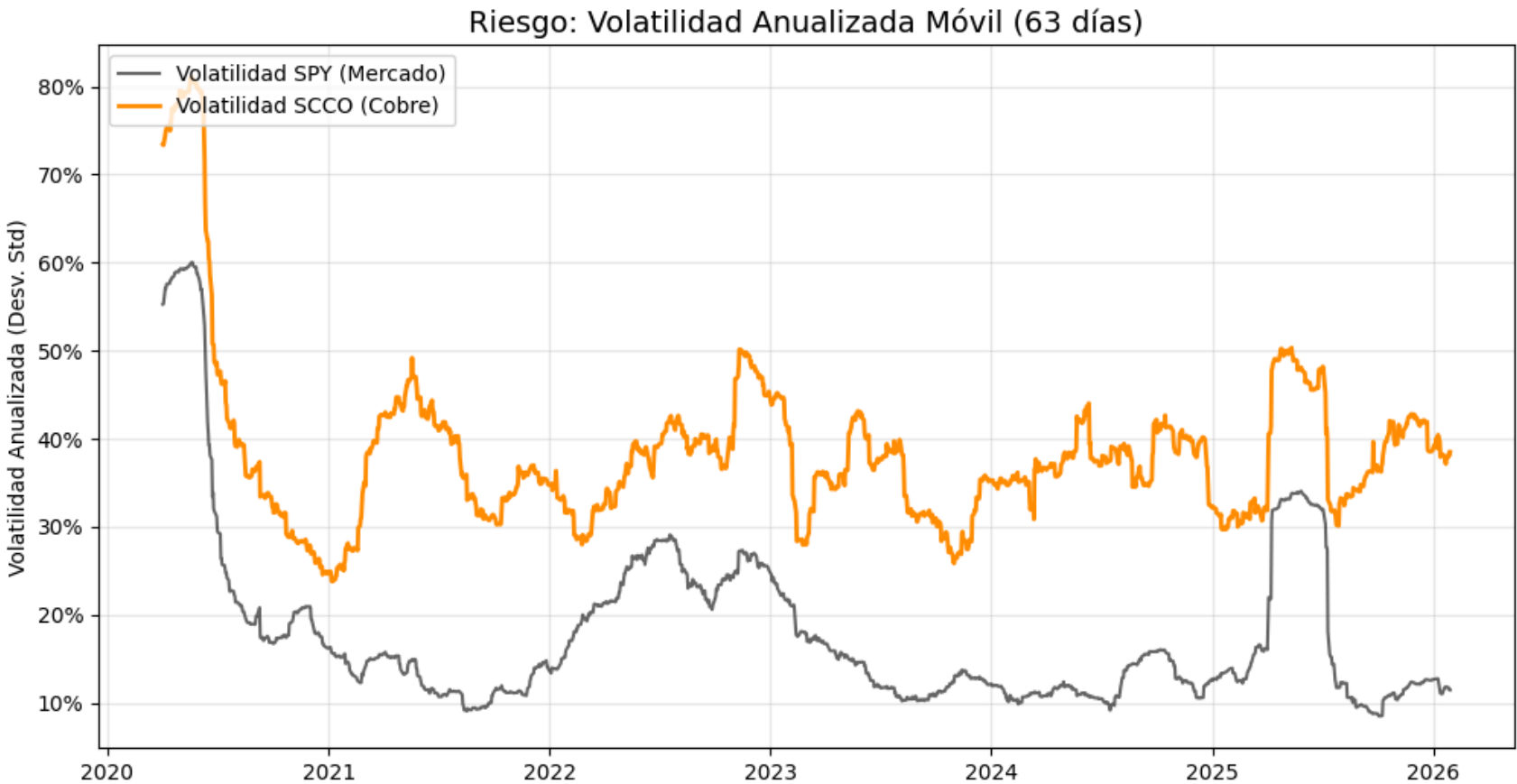


```
1 import matplotlib.pyplot as plt
2 # Usaremos 24 meses (2 años) para capturar tendencias de mediano plazo
3 window_size = 24
4
5 # 2. Calcular las Correlaciones Móviles
6
7 # A) Tasa Fed vs SPY (Costo del dinero vs Acciones)
8 roll_fed_spy = master_df['Tasa Fed (%)'].rolling(window_size).corr(master_df
9 ['SPY'])
10
11 # B) Inflación vs Oro (Cobertura histórica)
12 roll_cpi_gld = master_df['Inflación (CPI)'].rolling(window_size).corr(master_df
13 ['GLD'])
14
15 # C) Desempleo vs Cobre (Economía Real vs Materias Primas)
16 roll_unemp_scco = master_df['Desempleo (%)'].rolling(window_size).corr(master_df
17 ['SCCO'])
18
19 # 3. Graficar
20 plt.figure(figsize=(12, 6))
21
22 # Trazar las líneas
23 plt.plot(roll_fed_spy, label='Tasa Fed vs SPY', linewidth=2, color='red')
24 plt.plot(roll_cpi_gld, label='Inflación vs Oro (GLD)', linewidth=2,
25 color='gold')
26 plt.plot(roll_unemp_scco, label='Desempleo vs Cobre (SCCO)', linewidth=2,
27 color='green')
28
29 # Estética
30 plt.axhline(0, color='black', linestyle='--', alpha=0.5) # Línea cero de
31 referencia
32 plt.title(f'Evolución de Correlaciones (Ventana Móvil {window_size} meses)',
33 fontsize=14)
34 plt.ylabel('Coeficiente de Correlación (-1 a 1)')
35 plt.legend(loc='lower left')
36 plt.grid(True, alpha=0.3)
37
38 # Ajustar límites para ver bien el detalle
39 plt.ylim(-1.1, 1.1)
40
41 plt.show()
```



```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # 1. Calcular Retornos Diarios (Usamos los datos diarios originales de Yahoo)
5 daily_returns = market_data.pct_change().dropna()
6
7 # 2. Definir ventana de volatilidad (63 días = 1 Trimestre fiscal aprox)
8 vol_window = 63
9
10 # 3. Calcular Desviación Estándar Móvil y Anualizar
11 rolling_vol = daily_returns.rolling(window=vol_window).std() * np.sqrt(252)
12
13 # 4. Graficar Comparativa de Riesgo
14 plt.figure(figsize=(12, 6))
15
16 # Graficamos SPY (Mercado) vs SCCO (Minera)
17 plt.plot(rolling_vol['SPY'], label='Volatilidad SPY (Mercado)', color='black', alpha=0.6, linewidth=1.5)
18 plt.plot(rolling_vol['SCCO'], label='Volatilidad SCCO (Cobre)', color='darkorange', linewidth=2)
19
20 # Estética
21 plt.title(f'Riesgo: Volatilidad Anualizada Móvil ({vol_window} días)', fontsize=14)
22 plt.ylabel('Volatilidad Anualizada (Desv. Std)')
23 plt.legend(loc='upper left')
24 plt.grid(True, alpha=0.3)
25
26 # Formato de porcentaje en eje Y
27 vals = plt.gca().get_yticks()
28 plt.gca().set_yticklabels(['{:,.0%}'.format(x) for x in vals])
29
30 plt.show()
31
32 # 5. Imprimir Volatilidad Actual
33 print("Volatilidad Actual (Anualizada)")
34 print(rolling_vol.iloc[-1].round(4) * 100)
```

/tmp/ipython-input-2516939165.py:28: UserWarning: set_ticklabels() should only be used with a fixed number
plt.gca().set_yticklabels(['{:,.0%}'.format(x) for x in vals])



Volatilidad Actual (Anualizada)
Ticker
GLD 20.75
SCCO 38.49
SPY 11.47